1. What is Object Oriented Programming? Differentiate procedure oriented and object oriented programming language. OR List out characteristics of POP and OOP.

- Object Oriented Programming is programming paradigm that represents concepts as objects that has datafields and associated procedures known as methods.
- Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism etc. in programming.
- The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.

Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.

Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#, etc.

2. Explain Basic Concepts OOP. OR Explain Following terms.

Various concepts present in OOP to make it more powerful, secure, reliable and easy.

Object

- An object is an instance of a class.
- An object means anything from the real world like a person, computer, bench etc...
- Every object has at least one unique identity.
- An object is a component of a program that knows how to interact with other pieces of the program.
- An object is the variable of the type class.
- For example, If water is class then river is object.

Class

- A class is a **template** that specifies the **attributes** and **behavior** of things or objects.
- A class is a **blueprint** or **prototype** from which objects are created.
- A class is the implementation of an abstract data type (ADT).
- It defines **attributes** and **methods**

Abstraction

- Just represent essential features without including the background details.
- For example phone call, we don't know the internal processing.
- They encapsulate all the essential properties of the objects that are to be created.
- The attributes are sometimes called 'Data members' because they hold information.
- In Java, we use abstract class and interface to achieve abstraction.
- It is used to implement in class to provide data security.

Encapsulation

- **Wrapping up of a data and functions into single unit** is known as encapsulation.
- Only those functions can access it which is wrapped together within single unit.
- For example, a capsule, it is wrapped with different medicines.
- A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Inheritance

- Inheritance is the process, by which class can acquire the properties and methods of another class.
- The mechanism of deriving a new class from an old class is called inheritance.
- The new class is called **derived class** and old class is called **base class.**
- The derived class may have all the features of the base class.
- Programmer can add new features to the derived class.
- For example, Student is a base class and Result is derived class.
- It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

- A Greek word Polymorphism means the ability to take more than one form.
- Polymorphism allows a single name to be used for more than one related purpose.
- That means using a generic interface for a group of related activities.
- The advantage of polymorphism is that it helps to reduce complexity by allowing one interface to specify ageneral class of action'.
- For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.
- In Java, we use method overloading and method overriding to achieve polymorphism.
- Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.
- It means ability of operators and functions to act differently in different situations

3. List out benefits of OOP.

1. Troubleshooting is easier with the OOP language

- Suppose the user has no idea where the bug lies if there is an error within the code.
- Also, the user has no idea where to look into the code to fix the error.
- This is quite difficult for standard programming languages.
- However, when Object- Oriented Programming is applied, the user knows exactly where to look into the code whenever there is an error.
- There is no need to check other code sections as the error will show where the trouble lies.

2. Code Reusability

- Through inheritance, the same attributes of a class are not required to be written repeatedly.
- This avoids the issues where the same code has still to be written multiple times in a code.
- Thus, Object-Oriented Programming offers the feature of class reusability where the class that is once created can be used again.
- In doing so, time is saved, and the need for extra coding is eliminated as similar features can be inherited.

3. Productivity

- This is because the OOP has provided so many libraries that new programs have become more accessible.
- Also, as it provides the facility of code reusability, the length of a code is decreased, further enhancing the faster development of newer codes and programs.

4. Data Redundancy

- By the term data redundancy, it means that the data is repeated twice.
- This means that the same data is present more than one time.
- In Object-Oriented programming the data redundancy is considered to be an advantage.
- For example, the user wants to have a functionality that is similar to almost all the

classes.

- In such cases, the user can create classes with similar functionaries and inherit them wherever required. It reduces the repetition of a mundane task.

5. Code Flexibility

- The flexibility is offered through the concept of Polymorphism.
- A person can behave differently whenever the surroundings change.
- For example, if the person is in a market, the person will behave like a customer, or the behavior might get changed to a student when the person is in a school or any institution.
- In this example, it can be observed that different behaviors are shown by the same person whenever the surroundings around the person get changed. This could explain the concept of Polymorphism and its flexibility.
- The developers benefit through Polymorphism in the following ways: simplicity and extensibility.
- Polymorphism is one of the benefits of oop as it gives scope to a code to be in more than one form.

6. Solving problems

- Problems can be efficiently solved by breaking down the problem into smaller pieces and this makes as one of the big advantages of object-oriented programming.
- If a complex problem is broken down into smaller pieces or components, it becomes a good programming practice.

7. Security

- Because of the concept of data abstraction in OOPS, only a limited amount of data is shown to the user which makes good benefits of oop.
- The rest data is not exposed while exposing only the required amount of data.
- Therefore, it allows the maintenance of security.
- It also helps in avoiding repetitive code.
- Another concept provided in OOPS is the feature of encapsulation that allows the protection of the data in the classes from getting accessed by the system.
- All the internal contents in the class can be safeguarded.
- In Java, encapsulation is mainly used for restricting access to the class fields directly

while setting all the fields of the class to private.

8. Users can build programs from the standard modules that communicate with each other. This prevents writing the code again from scratch, which ultimately saves the development time of the code and therefore is higher productivity of the program.

9. Problems can be solved easily through OOPS because a program can be broken down into bit-sized codes or problems that can be easily solved.

10. With the development of technology, the maintenance cost of the programs is reduced, and there is increased productivity.

11. The upgrade of the OOP system of languages can be easily done from smaller systems to larger systems.

4. List out Applications of OOP.

Applications of OOP technology has gained importance in almost all areas of computing including real-time business systems.

Some of the examples are as follows:

1. Real-time systems
2. Simulation and modeling
3. Object oriented database
4. Artificial intelligence and expert systems
5. Neural networks and parallel programming
6. Decision support and office automation
7. CIM/CAM/CAD systems.
8. Distributed/Parallel/Heterogeneous computing
9. Data warehouse and Data mining/Web mini

5. What are syntax errors (compile errors), runtime errors, and logic errors?

1. Run Time Error:

- Run Time errors occur or we can say, are detected during the execution of the program.
- Sometimes these are discovered when the user enters an invalid data or data which is not relevant.
- Runtime errors occur when a program does not contain any syntax errors but asks the computer to do something that the computer is unable to reliably do.
- During compilation, the compiler has no technique to detect these kinds of errors.
- It is the JVM (Java Virtual Machine) that detects it while the program is running.
- To handle the error during the run time we can put our error code inside the try block and catch the error inside the catch block.
- **For example:** if the user inputs a data of string format when the computer is expecting an integer, there will be a runtime error. **Example 1:** Runtime Error caused by dividing by zero

2. Compile Time Error:

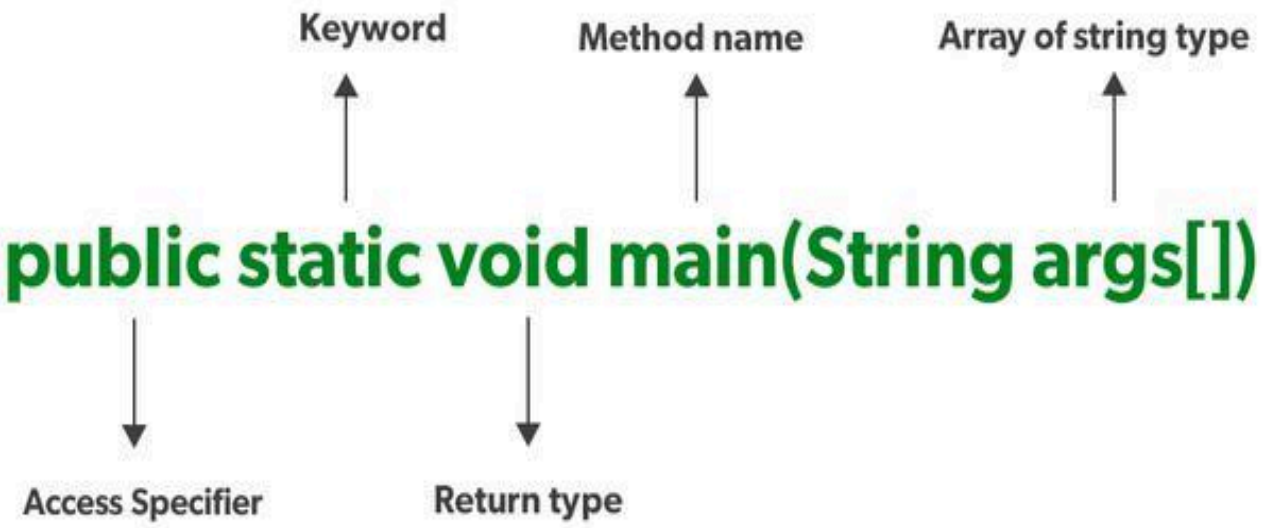
- Compile Time Errors are those errors which prevent the code from running because of an incorrect syntax such as a missing semicolon at the end of a statement or a missing bracket, class not found, etc.
- These errors are detected by the java compiler and an error message is displayed on the screen while compiling.
- Compile Time Errors are sometimes also referred to as **Syntax errors**.
- These kind of errors are easy to spot and rectify because the java compiler finds them for you.
- The compiler will tell you which piece of code in the program got in trouble and its best guess as to what you did wrong.
- Usually, the compiler indicates the exact line where the error is, or sometimes the line just before it, however, if the problem is with incorrectly nested braces, the actual error may be at the beginning of the block.
- In effect, syntax errors represent grammatical errors in the use of the programming language.
- **Example 1:** Misspelled variable name or method names

3. Logical Error:

- A logic error is when your program compiles and executes, but does the wrong thing or returns an incorrect result or no output when it should be returning an output.
- These errors are detected neither by the compiler nor by JVM.

6. Method main is a public static method. Justify

- The point from where the program starts its execution or simply the entry point of Java programs is the **main()** method.
- The Java compiler or JVM looks for the main method when it starts executing a Java program.
- The signature of the main method needs to be in a specific way for the JVM to recognize that method as its entry point.
- If we change the signature of the method, the program compiles but does not execute.
- The execution of the Java program, the ***java.exe*** is called.
- The Java.exe in turn makes Java Native Interface or JNI calls, and they load the JVM.
- The java.exe parses the command line, generates a new String array, and invokes the main() method.
- A daemon thread is attached to the main method, and this thread gets destroyed only when the Java program stops execution.



1. Public

- It is an ***Access modifier***, which specifies from where and who can access the method.
- Making the *main()* method public makes it globally available.

- It is made public so that JVM can invoke it from outside the class as it is not present in the current class.

2. Static

- It is a ***keyword*** that is when associated with a method, making it a class-related method.
- The *main()* method is static so that JVM can invoke it **without instantiating** the class.
- This also saves the unnecessary wastage of memory which would have been used by the object declared only for calling the *main()* method by the JVM.

3. Void

- It is a keyword and is used to specify that a method doesn't return anything.
- As the *main()* method doesn't return anything, its return type is *void*.
- As soon as the *main()* method terminates, the java program terminates too.
- Hence, it doesn't make any sense to return from the *main()* method as JVM can't do anything with the return value of it.

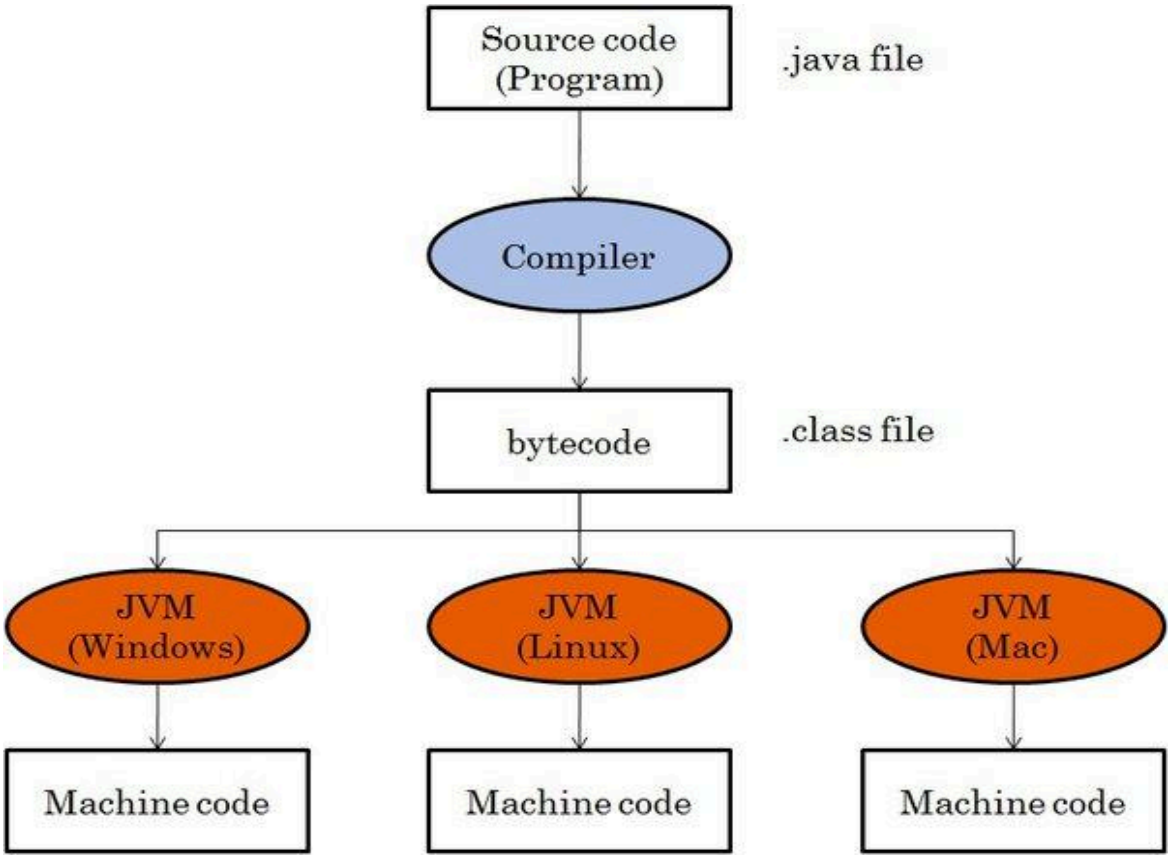
4. main

- It is the name of the Java main method.
- It is the identifier that the JVM looks for as the starting point of the java program.
- It's not a keyword.

5. String[] args

- It stores Java command-line arguments and is an array of type ***java.lang.String*** class.
- Here, the name of the String array is *args* but it is not fixed and the user can use any name in place of it.

7. Significance of Bytecode



- The feature that makes java *platform independent* is "*Java Byte Code*".
- Java bytecode is the instruction set for the Java Virtual Machine.
- Every platform has its corresponding JVM.
- Now the JVMs accept this unique byte code generated by the java compiler and convert them into the native machine code which is unique for each platform.
- These machine codes are later executed.
- In **Java , Bytecode** is a set of instruction that is run by **JVM(Java Virtual Machine).**
- This ByteCode distincts Java from other languages like C,C++.
- That's why Java is platform Independent, meaning , it can be run on any Operating System.
- So what happens is, in Java the compiler Javac converts your .java file to bytecode under .class file and then the Interpreter built under JVM converts this byteCode to machine code required on that Machine.
- *Bytecode is a portable form of executable code.*
- *Bytecode is platform independent*
- Byte code generated is basically represented in hexa decimal format.
- This format is same on every platform, be it Mac, windows or Linux.

- After compilation, the interpreter reads the generated byte code & translates it according to the host machine.
- Byte code is interpreted by Java Virtual Machine which is available with all the operating systems we install.

8.Write a single program which demonstrates the usage of following keywords:

i) import, ii) new .

And Show how to compile and run the program in java.

Java Program: Using `import` and `new`

```
// Importing Scanner class from java.util package
    import java.util.Scanner;
    class KeywordDemo
    {
        public static void main(String[] args)
        {
        // Using 'new' keyword to create Scanner object
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter your name: ");
            String name = sc.nextLine();
            System.out.println("Hello, " + name);
        }
    }
```

Output:

Enter your name: Bhavisha

Hello, Bhavisha

- ☐ **`import` keyword:** Used to **access predefined classes** from Java packages
- ☐ `import java.util.Scanner;` (allows us to use the `Scanner` class for input.)

🔹 **new keyword:** Used to **create an object (instance) of a class**.

Here:

```
Scanner sc = new Scanner(System.in);
```

- creates an object `sc` of the `Scanner` class.

how to compile and run the program in java.

Compile the Program

```
javac KeywordDemo.java
```

✅ If no error appears, compilation is successful

✅ A file `KeywordDemo.class` will be created

Run the Program

```
java KeywordDemo
```

9. Explain class and object with respect to java.

Class in Java

Definition:

- ☐ A class is a blueprint or template that defines properties (variables) and behaviors (methods) of an object.
- ☐ A class is a group of objects which have common properties.
- ☐ It is a logical entity.
- ☐ It can't be physical.

A class in Java can contain:

- Fields
- Methods
- Constructors
- Blocks
- Nested class and interface

Note: A class does not occupy memory until its object is created.

Real-Life Example

Think of a class as a blueprint of a house 🏠

- Blueprint → design only

- No real house exists until it is built

Syntax of Class

```
class ClassName
{
   // data members
   // methods
}
```

Object in Java

Definition: An object is a real instance of a class.

- ☐ Object occupies memory
- ☐ Object is created using the new keyword
- ☐ An object is a real-world entity.
- ☐ An object is a runtime entity.
- ☐ The object is an entity which has state and behavior.

Real-Life Example

- House built using blueprint = Object
- Student class → Ravi, Anjali = Objects

Syntax of Object

```
ClassName obj = new ClassName();
```

Example Program (Class & Object)

```
class Student
{
        int id;
        String name;
        void display()
        {
                System.out.println(id + " " + name);
        }
    public static void main(String[] args)
    {
        // Creating object of Student class using new keyword
         Student s1 = new Student();
        s1.id = 101;
        s1.name = "Ravi";
        s1.display();
    }
}
```

Output:

```
101 Ravi
```

10. Method in Java or
What is purpose of using methods?
How do you declare a method?
How do you invoke a method?

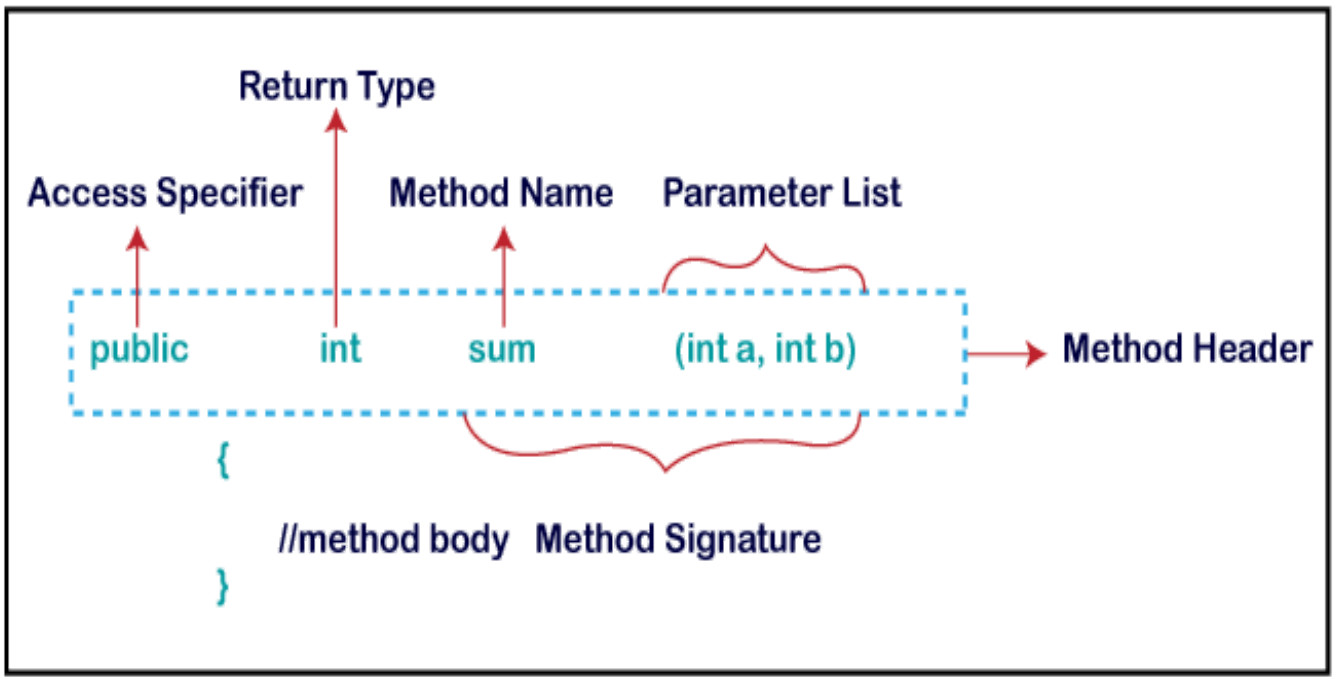
- ☐ A method is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation.
- ☐ A **method** is a block of code that performs a **specific task**.

Purpose of Using Methods in Java

- ☐ ✅ **Code reusability** – write once, use many times
- ☐ ✅ **Reduces code duplication**
- ☐ ✅ **Improves readability & structure**
- ☐ ✅ **Easy debugging and maintenance**
- ☐ ✅ **Breaks large programs into smaller parts**

- ☐ It is used to achieve the reusability of code.
- ☐ We write a method once and use it many times.
- ☐ We do not require to write code again and again.
- ☐ It also provides the easy modification and readability of code, just by adding or removing a chunk of code.
- ☐ The method is executed only when we call or invoke it.
- ☐ The most important method in Java is the main() method

Method Declaration



- **Method Signature:** Every method has a method signature. It is a part of the method declaration. It includes the **method name** and **parameter list**.
- **Access Specifier:** Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides **four** types of access specifier:
- **public:** The method is accessible by all classes when we use public specifier in our application.
- **Private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **Protected:** When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **Default:** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.
- **Return Type:** Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use void keyword.
- **Method Name**: It is a unique name that is used to define the name of a method. It must be corresponding to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be subtraction(). A method is invoked by its

name.

- **Parameter List:** It is the list of parameters separated by a comma and enclosed in the pair of parentheses. It contains the data type and variable name. If the method has no parameter, left the parentheses blank.
- **Method Body:** It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within the pair of curly braces.

How to Declare a Method in Java

General Syntax:

```
returnType methodName(parameter_list)
{
   // method body
}
```

Example:

```
int add(int a, int b)
{
   return a + b;
}
```

- int → return type
- add → method name
- int a, int b → parameters
- return → sends value back to caller

How to Invoke (Call) a Method

Syntax:

```
objectName.methodName(arguments);
```

Example:

```
class MethodDemo
{

        // Method declaration
            int add(int x, int y)
            {
                    return x + y;
            }

        public static void main(String[] args)
        {

                MethodDemo obj = new MethodDemo();  // Object creation

                int result = obj.add(10, 20); // Method invocation

                System.out.println("Sum = " + result);

        }
}
```

Output:

Sum = 30

11.Types of Method

1. Predefined(Built-in) Method

- In Java, predefined methods are the method that is already defined in the Java class libraries and is known as predefined methods.
- It is also known as the **standard library method** or **built-in method**.

Examples:

- `System.out.println()`
- `Math.sqrt()`

- ☐ `String.length()`

Example:

```
public class Demo
{
    public static void main(String[] args)
    {
        // using the max() method of Math class
        System.out.print("The maximum number is: " + Math.max(9,7));
    }
}
```

Output: The maximum number is: 9

2. User-Defined Methods

- ☐ The method written by the user or programmer is known as a user-defined method.
- ☐ Used to perform specific tasks

Examples:

- ☐ `System.out.println()`
- ☐ `Math.sqrt()`
- ☐ `String.length()`

Example:

```
public class Demo
{
    public static void main(String[] args)
    {
        // using the max() method of Math class
```

```
        System.out.print("The maximum number is: " + Math.max(9,7));
    }
}
```

Output: The maximum number is: 9

12. Compare static methods with instance methods.

Feature	Static Methods	Instance Methods
Belongs to	The class itself	A specific object (instance)
Invocation	Called using the class name (e.g., **ClassName.method()**)	Called using an object reference (e.g., **object.method()**)
Object Required?	**No**, they can be called without creating an instance	**Yes**, an object must be created first
Access to Members	Can only access **static variables and other static methods** directly	Can access **both static and instance variables/methods**
this Keyword	**Cannot** use the **this** or **super** keywords	Can use this to refer to the current object and super for the parent class
Polymorphism	**Cannot be overridden** in subclasses (they are "hidden" instead)	Support **runtime polymorphism** (can be overridden)
Memory	A single copy exists per class, shared by all instances	Each object has its own copy of the methods (conceptually, though code is shared in memory, the method call is tied to an object's state)

13. Data Encapsulation by defining classes

- Encapsulation in Java is a mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit.

- In encapsulation, a class's variables are hidden from other classes and can only be accessed by the methods of the class in which they are found.
- This also helps to achieve data hiding.

Example of Data Encapsulation

```
class Person {
  private String name = "Amit";  // private data
  public String getName() {     // public method
    return name;
  }
}
public class Main {
  public static void main(String[] args) {
    Person p = new Person();
    System.out.println(p.getName());
  }
}
```

Note: The data `name` is private and accessed using one public method, so encapsulation is achieved.

Advantages of Encapsulation

The advantages of encapsulation are listed below:

- **Data Hiding:** Encapsulation restricts direct access to class variables, protecting sensitive data from unauthorized access.

- **Improved Maintainability:** Changes to internal implementation can be made without affecting external code that uses the class.
- **Enhanced Security:** Encapsulation allows validation and control over data, preventing invalid or harmful values from being set.
- **Code Reusability:** Encapsulated classes can be reused in different programs without exposing internal logic.
- **Better Modularity:** Encapsulation promotes organized, modular code by keeping data and methods together within a class.

Define constructor. How objects are constructed? Explain

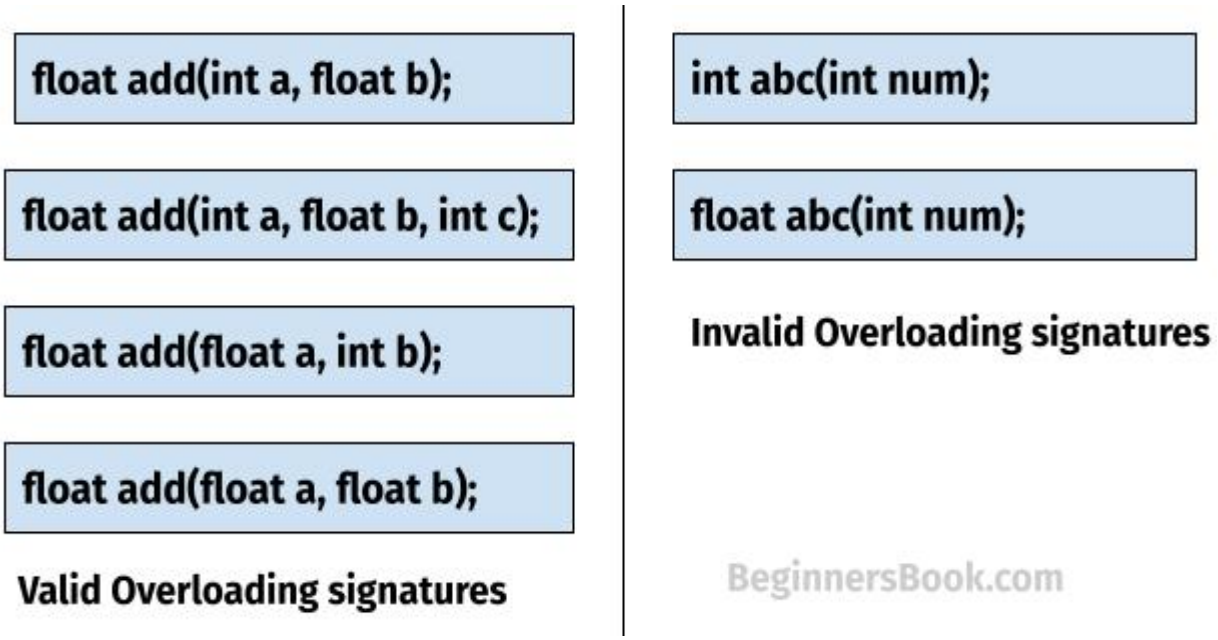
14. Give definition of method overloading. Or

Explain Method overloading with example.

- ☐ **Method Overloading is a feature that allows a class to have multiple methods with the same name but with different number, sequence or type of parameters.**
- ☐ **In short multiple methods with same name but with different signatures.**
- ☐ For example the signature of method add(int a, int b) having two int parameters is different from signature of method add(int a, int b, int c) having three int parameters.

Method overloading means:

- ☐ **Same method name**
- ☐ **Different parameters (number or data type)**
- ☐ **In the same class**



Three ways to overload a method

1. **Number of parameters.**

   add(int, int)

   add(int, int, int)

2. **Data type of parameters.**

   add(int, int)

   add(int, float)

3. **Sequence of Data type of parameters.**

   add(int, float)

   add(float, int)

Example:

```
class Addition
{
  // Method 1: add two integers
        void add(int a, int b)
        {
```

```
                System.out.println("Addition of 2 ints: " + (a + b));
        }
    // Method 2: add three integers
        void add(int a, int b, int c)
        {
                System.out.println("Addition of 3 ints: " + (a + b + c));
        }
    // Method 3: add two double values
        void add(double a, double b)
        {
                System.out.println("Addition of 2 doubles: " + (a + b));
        }
    public static void main(String[] args)
    {
        Addition obj = new Addition();
        obj.add(10, 20);
        obj.add(5, 10, 15);
        obj.add(2.5, 3.5);
    }
}
```

Output

```
Addition of 2 ints: 30
Addition of 3 ints: 30
Addition of 2 doubles: 6.0
```

15. What is constructor?
What is it role?
Explain various features/characteristics of constructors.

- ☐ In Java, a constructor is a block of codes similar to the method.
- ☐ A **constructor** is a **special method of a class** that is **automatically called when an object is created**. Its main purpose is to **initialize (give initial values to) the data members of a class**.
- ☐ Constructor name is **same as the class name**.
- ☐ At the time of calling the constructor, memory for the object is allocated in the memory.
- ☐ Every time an object is created using the new() keyword, at least one constructor is called.
- ☐ It calls a default constructor if there is no constructor available in the class.
- ☐ In such case, Java compiler provides a default constructor by default.
- ☐ There are two types of constructors in Java: **no-arg constructor, and parameterized constructor.**
- ☐ It is called a constructor because it constructs the values at the time of object creation.

What is the Role of a Constructor?

The role of a constructor is to:

- Initialize data members of a class
- Allocate memory to an object
- Set default or initial values when an object is created
- Ensure the object is in a valid state before use

Features / Characteristics of Constructors

1 Same Name as Class: Constructor name must be exactly the same as the class name.

```
class Demo {
    Demo() { }
}
```

2 No Return Type: Constructors do not have any return type, not even `void`.

3 **Automatically Invoked:** Constructor is **called automatically** when an object is created using `new`.

```
Demo d = new Demo();
```

4 **Used for Initialization:** Constructors initialize instance variables.

```
Demo(int x) {
    a = x;
}
```

5 **Can Be Overloaded:** Multiple constructors can exist in a class with **different parameters**.

```
Demo() { }
Demo(int a) { }
```

6 **Cannot Be Inherited:** Constructors **are not inherited** by subclasses.

7 **Can Have Access Modifiers:** Constructors can be:

`Public, private, protected,` default

8 **Default Constructor:** If no constructor is written, **Java provides a default constructor automatically**.

```
Demo() { }
```

16.Explain constructor with the help of an example.

- ☐ A constructor is a special **method** of a class that is **automatically executed** when an **object** is **created**.
- ☐ It is mainly used to **initialize** the **data members of a class.**

Key Points

- Constructor name is **same as the class name**
- It **has no return type**
- It is called **automatically** when an object is created using new

Example:

```
class Student

{

        int rollNo;

        String name;

  // Constructor

  Student(int r, String n) {

    rollNo = r;

    name = n;

  }

 void display()

{

    System.out.println(rollNo + " " + name);

  }
```

```
    public static void main(String args[]) {

      Student s1 = new Student(101, "Bhavisha"); // constructor called

      s1.display();

    }

  }
```

Explanation of the Example:

- `Student(int r, String n)` is a **constructor**
- It initializes `rollNo` and `name`
- When the object `s1` is created using `new Student(101, "RAHUL")`, the constructor is **called automatically**
- The initialized values are then displayed

Output

```
101 RAHUL
```

17. constructor overloading with an example.

- ☐ Constructor overloading is a technique in object-oriented programming (Java, C++, C#) where a **class has multiple constructors with the same name but different parameter lists (type, number, or order).**
- ☐ It enables initializing objects in different ways, such as using default values or custom data.

Why Use Constructor Overloading?

- ☐ To give flexibility when creating objects
- ☐ To set default or custom values
- ☐ To reduce repetitive code

Key points:

Constructor overloading means

👉 **having more than one constructor in the same class**,

👉 **with different parameter lists** (number or type of parameters).

Example 1: Constructor Overloading in Java

```
class Student
{
    int id;
    String name;
    Student()   // 1. Default Constructor (No arguments)
    {
        id = 0;
        name = "Unknown";
    }
    Student(int i, String n)   // 2. Parameterized Constructor (Two arguments)
    {
        id = i;
        name = n;
    }
    void display()
    {
```

```
                System.out.println(id + " " + name);
            }
        public static void main(String[] args)
        {
                Student s1 = new Student();   // Calls constructor 1
                Student s2 = new Student(101, "Alice"); // Calls constructor 2
                s1.display();    // Output: 0 Unknown
                s2.display();   // Output: 101 Alice
        }
    }
```

Example 2: Constructor Overloading

```
class Box
{
    double width, height, depth;
    Box(double w, double h, double d)
    {
      width = w;
      height = h;
      depth = d;
    }
    Box() // No-argument constructor
    {
      width = height = depth = 0;
    }
    Box(double len) // Constructor for cube
    {
      width = height = depth = len;
    }
    double volume()
    {
      return width * height * depth;
    }

  public static void main(String[] args)
```

```
        {
            Box mybox1 = new Box(10, 20, 15);
            Box mybox2 = new Box();
            Box mycube  = new Box(7);
            System.out.println("Volume of mybox1: " + mybox1.volume());
            System.out.println("Volume of mybox2: " + mybox2.volume());
            System.out.println("Volume of mycube: " + mycube.volume());
        }
    }
```

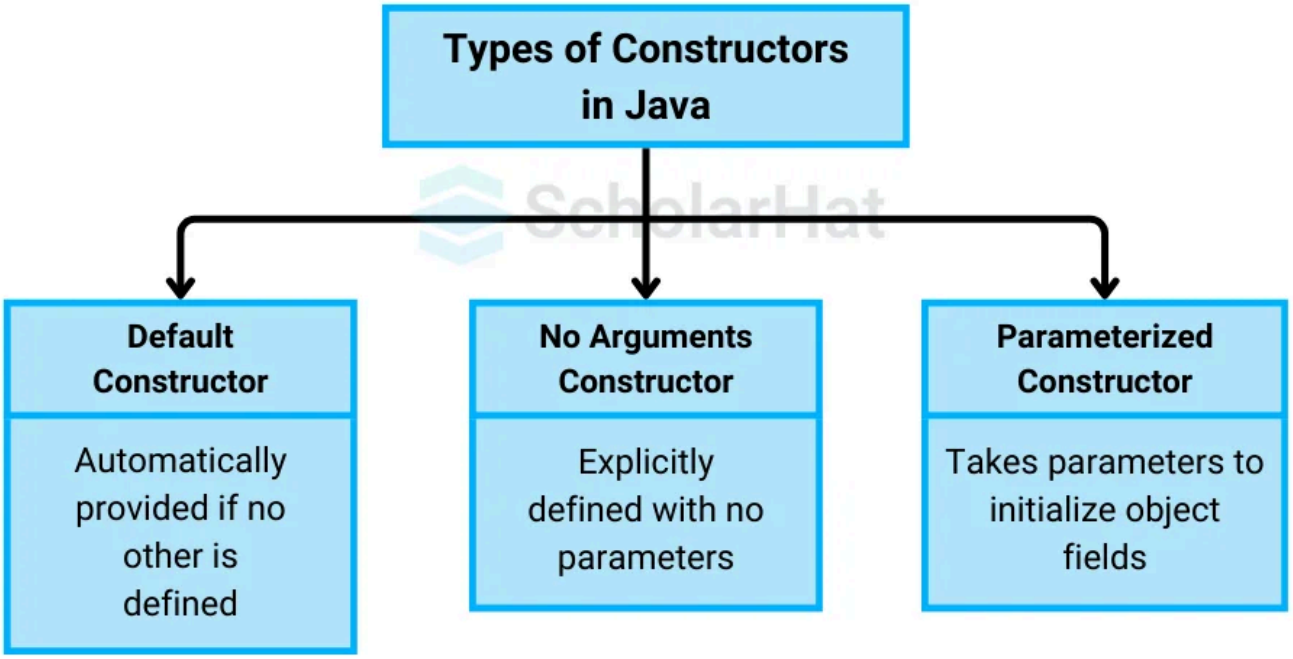
Output:

```
Volume of mybox1: 3000.0

Volume of mybox2: 0.0

Volume of mycube: 343.0
```

18. List and explain available type of constructors in java with example.



1) Default Constructor

- ☐ A constructor **with no parameters** is called Default Constructor
- ☐ If you don't write any constructor, **Java provides it automatically**
- ☐ It initializes data members with **default values**

Example:

```
class Student
{
        int roll;
        String name;
   public static void main(String args[])
   {
            Student s = new Student();
            System.out.println(s.roll);   // 0
            System.out.println(s.name);   // null
   }
}
```

What is happening?

- Java sees **no constructor**
- Java provides a **default constructor**
- Variables get default values

2) No-args Constructor

- ☐ **A no-argument (no-args) constructor** is a constructor that is **explicitly defined by the programmer**
- ☐ It does not take any parameters, and it is used to initialize an object with default or

programmer-defined values at the time of object creation.

Example:

```
class Student
{
        int roll;
        String name;
        Student()   // No-args Constructor
        {
                roll = 10;
                name = "Amit";
        }
  public static void main(String args[])
  {
            Student s = new Student();
            System.out.println(s.roll);   // 10
            System.out.println(s.name);   // Amit
  }
}
```

What is happening?

- ☐ Constructor is **explicitly written**
- ☐ Custom values are assigned
- ☐ Java **does NOT** provide default constructor now

3) Parameterized Constructor

- ☐ A constructor with **parameters**
- ☐ Used to **initialize objects with user-defined values**

Example:

```
class Student
{
        int roll;
        String name;
        Student(int r, String n)  // Parameterised Constructor
        {
                roll = r;
                name = n;
        }

  public static void main(String args[])
  {
                Student s = new Student(1, "Ravi");
                System.out.println(s.roll);
                System.out.println(s.name);
  }
}
```

19. Difference Between Constructor and Method.

Constructors	Methods
A Constructor is a block of code that **initializes a newly created object.**	A Method is a **collection of statements** which returns a value upon its execution.
A Constructor can be used to **initialize an object.**	A Method consists of Java **code to be executed**.
A Constructor is **invoked implicitly** by system.	A Method is **invoked by the programmer.**

A Constructor is invoked when a object is created using the keyword **new**.	A Method is invoked through method calls.
A Constructor **doesn't have a return type.**	A Method **must have a return type.**
A Constructor initializes a object that doesn't exist.	A Method does operations on an already created object.
A Constructor's **name must be same as the name of the class.**	A Method's **name can be anything.**

Syntax of Constructor
ClassName()
{
// initialization code
}
	Syntax of Method
	returnType methodName(parameters)
	{
	// method body
	}

Example of Constructor
class Student
{
Student()
{ // constructor
System.out.println("Constructor called");
}
}
	Example of Method
	class Student
	{
	Void display()
	{
	System.out.println("Method called");
	}
	}

A Constructor cannot be inherited by subclasses.	A Method can be inherited by subclasses.

20. How to access object via reference variable? Explain with example.

- In Java, **an object is accessed using its reference variable**.
- The reference variable stores the **memory address of the object**, and using the **dot (.) operator**, we can access the **data members and methods** of that object.

Syntax:

```
referenceVariable.dataMember;

referenceVariable.methodName();
```

Example:

```
class Student

{

    int roll;

    String name;

  void display()

  {

      System.out.println(roll);

      System.out.println(name);
```

```
    }

    public static void main(String args[])

        {

            Student s = new Student();   // s is reference variable

            s.roll = 101;            // accessing data member

            s.name = "Ravi";

            s.display();             // accessing method

    }

}
```

Output:

```
101

Ravi
```

Explanation

1. `Student s = new Student();`

   → Object is created

   → `s` stores the reference (address) of the object

2. `s.roll` and `s.name`

   → Access object's variables using reference variable

3. `s.display()`

   → Access object's method using reference variable

21.How objects are constructed?

- ☐ An object in Java is constructed when the **`new` keyword** is used with a **class constructor**.
- ☐ During object construction, **memory is allocated**, **instance variables are initialized**, and the **constructor is executed**.

Syntax:

```
ClassName referenceVariable = new ClassName();
```

Example:

```
class Student
{
        int roll;
        String name;
        Student()
        {
                roll = 1;
                name = "Rahul";
        }
   public static void main(String args[])
  {
```

```
        Student s = new Student();
    }
}
```

What happens internally?

1. Class is loaded
   - JVM loads the Student class into memory.
2. Memory allocation
   - new Student() allocates memory for the object in heap memory.
3. Reference creation
   - Reference variable s is created in stack memory.
4. Instance variables initialized
   - Variables get default values first (0, null).
5. Constructor is called
   - Student() constructor runs and assigns given values.
6. Reference linked to object
   - s now refers to the created object.